

10.1 Basic Pointer Operations

[This section corresponds to K&R Sec. 5.1]

The first things to do with pointers are to declare a pointer variable, set it to point somewhere, and finally manipulate the value that it points to. A simple pointer declaration looks like this:

```
int *ip;
```

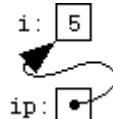
This declaration looks like our earlier declarations, with one obvious difference: that asterisk. The asterisk means that `ip`, the variable we're declaring, is not of type `int`, but rather of type pointer-to-`int`. (Another way of looking at it is that `*ip`, which as we'll see is the value pointed to by `ip`, will be an `int`.)

We may think of setting a pointer variable to point to another variable as a two-step process: first we generate a pointer to that other variable, then we assign this new pointer to the pointer variable. We can say (but we have to be careful when we're saying it) that a pointer variable has a value, and that its value is "pointer to that other variable". This will make more sense when we see how to generate pointer values.

Pointers (that is, pointer values) are generated with the "address-of" operator `&`, which we can also think of as the "pointer-to" operator. We demonstrate this by declaring (and initializing) an `int` variable `i`, and then setting `ip` to point to it:

```
int i = 5;  
ip = &i;
```

The assignment expression `ip = &i` contains both parts of the "two-step process": `&i` generates a pointer to `i`, and the assignment operator assigns the new pointer to (that is, places it "in") the variable `ip`. Now `ip` "points to" `i`, which we can illustrate with this picture:



`i` is a variable of type `int`, so the value in its box is a number, 5. `ip` is a variable of type pointer-to-`int`, so the "value" in its box is an arrow pointing at another box. Referring once again back to the "two-step process" for setting a pointer variable: the `&` operator draws us the arrowhead pointing at `i`'s box, and the assignment operator `=`, with the pointer variable `ip` on its left, anchors the other end of the arrow in `ip`'s box.

We discover the value pointed to by a pointer using the "contents-of" operator, `*`. Placed in front of a pointer, the `*` operator accesses the value pointed to by that pointer. In other words, if `ip` is a pointer, then the expression `*ip` gives us whatever it is that's in the variable or location pointed to by `ip`. For example, we could write something like

```
printf("%d\n", *ip);
```

which would print 5, since `ip` points to `i`, and `i` is (at the moment) 5.

(You may wonder how the asterisk `*` can be the pointer contents-of operator when it is also the multiplication operator. There is no ambiguity here: it is the multiplication operator when it sits between two variables, and it is the contents-of operator when it sits in front of a single variable. The situation is analogous to the minus sign: between two variables or expressions it's the subtraction

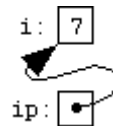
operator, but in front of a single operator or expression it's the negation operator. Technical terms you may hear for these distinct roles are *unary* and *binary*: a *binary* operator applies to two operands, usually on either side of it, while a *unary* operator applies to a single operand.)

The contents-of operator `*` does not merely fetch values through pointers; it can also *set* values through pointers. We can write something like

```
*ip = 7;
```

which means "set whatever `ip` points to to 7." Again, the `*` tells us to go to the location pointed to by `ip`, but this time, the location isn't the one to fetch from--we're on the left-hand side of an assignment operator, so `*ip` tells us the location to store *to*. (The situation is no different from array subscripting expressions such as `a[3]` which we've already seen appearing on both sides of assignments.)

The result of the assignment `*ip = 7` is that `i`'s value is changed to 7, and the picture changes to:



If we called `printf("%d\n", *ip)` again, it would now print 7.

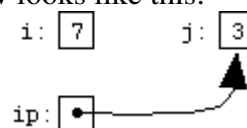
At this point, you may be wondering why we're going through this rigamarole--if we wanted to set `i` to 7, why didn't we do it directly? We'll begin to explore that next, but first let's notice the difference between changing a pointer (that is, changing what variable it points to) and changing the value at the location it points to. When we wrote `*ip = 7`, we changed the value pointed to by `ip`, but if we declare another variable `j`:

```
int j = 3;
```

and write

```
ip = &j;
```

we've changed `ip` itself. The picture now looks like this:



We have to be careful when we say that a pointer assignment changes "what the pointer points to." Our earlier assignment

```
*ip = 7;
```

changed the value pointed to by `ip`, but this more recent assignment

```
ip = &j;
```

has changed what *variable* `ip` points to. It's true that "what `ip` points to" has changed, but this time, it has changed for a different reason. Neither `i` (which is still 7) nor `j` (which is still 3) has changed. (What has changed is `ip`'s value.) If we again call

```
printf("%d\n", *ip);
```

this time it will print 3.

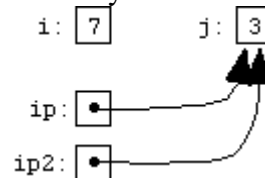
We can also assign pointer values to other pointer variables. If we declare a second pointer variable:

```
int *ip2;
```

then we can say

```
ip2 = ip;
```

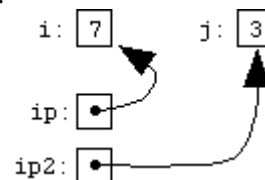
Now `ip2` points where `ip` does; we've essentially made a ``copy" of the arrow:



Now, if we set `ip` to point back to `i` again:

```
ip = &i;
```

the two arrows point to different places:



We can now see that the two assignments

```
ip2 = ip;
```

and

```
*ip2 = *ip;
```

do two very different things. The first would make `ip2` again point to where `ip` points (in other words, back to `i` again). The second would store, at the location pointed to by `ip2`, a copy of the value pointed to by `ip`; in other words (if `ip` and `ip2` still point to `i` and `j` respectively) it would set `j` to `i`'s value, or 7.

It's important to keep very clear in your mind the distinction between *a pointer* and *what it points to*. The two are like apples and oranges (or perhaps oil and water); you can't mix them. You can't ``set `ip` to 5" by writing something like

```
ip = 5;                /* WRONG */
```

5 is an integer, but `ip` is a pointer. You probably wanted to ``set *the value pointed to by* `ip` to 5," which you express by writing

```
*ip = 5;
```

Similarly, you can't ``see what `ip` is" by writing

```
printf("%d\n", ip);      /* WRONG */
```

Again, `ip` is a pointer-to-`int`, but `%d` expects an `int`. To print *what `ip` points to*, use

```
printf("%d\n", *ip);
```

Finally, a few more notes about pointer declarations. The `*` in a pointer declaration is related to, but different from, the contents-of operator `*`. After we declare a pointer variable

```
int *ip;
```

the expression

```
ip = &i
```

sets what `ip` points to (that is, which location it points to), while the expression

```
*ip = 5
```

sets the value of the location pointed to by `ip`. On the other hand, if we declare a pointer variable and include an initializer:

```
int *ip3 = &i;
```

we're setting the initial value for `ip3`, which is where `ip3` will point, so that initial value is a pointer. (In other words, the `*` in the declaration `int *ip3 = &i;` is not the contents-of operator, it's the indicator that `ip3` is a pointer.)

If you have a pointer declaration containing an initialization, and you ever have occasion to break it up into a simple declaration and a conventional assignment, do it like this:

```
int *ip3;  
ip3 = &i;
```

Don't write

```
int *ip3;  
*ip3 = &i;
```

or you'll be trying to mix oil and water again.

Also, when we write

```
int *ip;
```

although the asterisk affects `ip`'s type, it goes with the identifier name `ip`, not with the type `int` on the left. To declare two pointers at once, the declaration looks like

```
int *ip1, *ip2;
```

Some people write pointer declarations like this:

```
int* ip;
```

This works for one pointer, because C essentially ignores whitespace. But if you ever write

```
int* ip1, ip2;          /* PROBABLY WRONG */
```

it will declare one pointer-to-int `ip1` and one *plain* `int ip2`, which is probably not what you meant.

What is all of this good for? If it was just for changing variables like `i` from 5 to 7, it would not be good for much. What it's good for, among other things, is when for various reasons we don't know exactly which variable we want to change, just like the bank didn't know exactly which club member it wanted to send the statement to.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)